

ST10465374
PROG6212

Makutuma Piet Modiba
ROSEBANK COLLEGE braamfontein

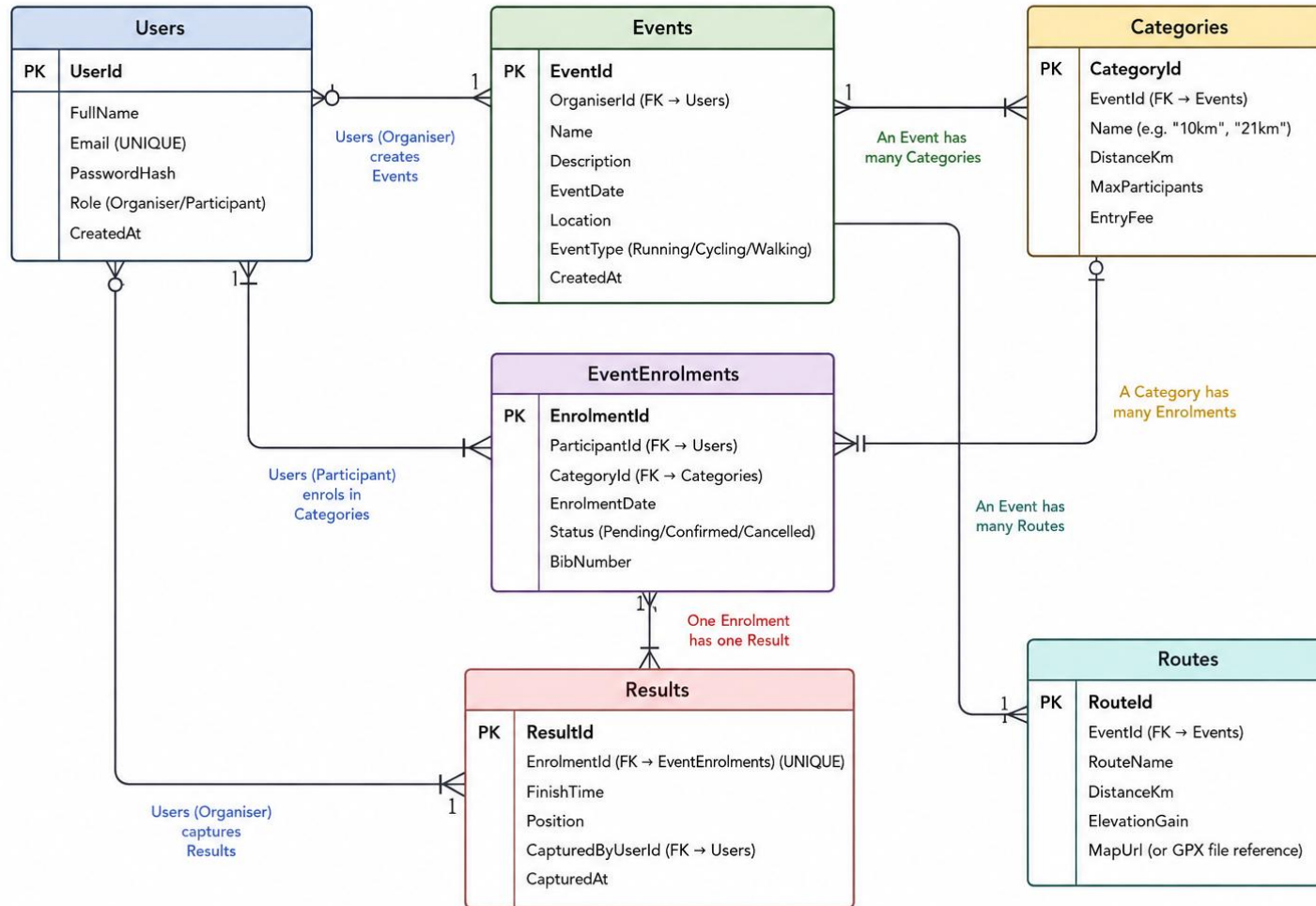
LINKS

YOUTUBE LINK: <https://youtu.be/KFbWPcO-EEI>

GITHUB LINK: <https://github.com/makutuma/RaceDay.git>

SECTION A

ERD DIAGRAM



SECTION B

RaceDay API

Endpoint Plan

Authentication

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Creates a new user account as either an Organiser or a Participant.	Public	{ fullName, email, password, role }	201 Created – user object (no password) · 400 Bad Request – validation failed · 409 Conflict – email already registered
POST	/api/auth/login	Authenticates a user and returns a JWT for subsequent requests.	Public	{ email, password }	200 OK – { token, userId, role } · 401 Unauthorized – invalid credentials

User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me	Returns the profile of the currently logged-in user.	Any (logged in)	None	200 OK – user profile object · 401 Unauthorized
PUT	/api/users/me	Updates the logged-in user's own profile details.	Any (logged in)	{ fullName, email }	200 OK – updated profile · 400 Bad Request · 401 Unauthorized

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/{id}	Retrieves a specific user's public profile (e.g. organiser contact info on an event page).	Any (logged in)	None	200 OK – public profile fields · 404 Not Found

Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Lists all upcoming events, with optional filters (type, location, date range).	Public	None	200 OK – array of events
GET	/api/events/{id}	Retrieves full details for a single event, including its categories and route.	Public	None	200 OK – event object · 404 Not Found
POST	/api/events	Creates a new event.	Organiser	{ name, description, eventDate, location, eventType }	201 Created – event object · 400 Bad Request · 401 Unauthorized
PUT	/api/events/{id}	Updates an event owned by the logged-in organiser.	Organiser	{ name, description, eventDate, location, eventType }	200 OK – updated event · 403 Forbidden (not owner) · 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
DELETE	/api/events/{id}	Deletes an event owned by the logged-in organiser.	Organiser	None	204 No Content · 403 Forbidden · 404 Not Found
GET	/api/events/{id}/enrolments	Lists all participants enrolled across an event's categories.	Organiser	None	200 OK – array of enrolments · 403 Forbidden · 404 Not Found

Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Lists all categories (e.g. 5km, 10km) for a specific event.	Public	None	200 OK – array of categories · 404 Not Found
POST	/api/events/{eventId}/categories	Adds a new category to an event owned by the organiser.	Organiser	{ name, distanceKm, maxParticipants, entryFee }	201 Created – category object · 403 Forbidden · 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
PUT	/api/categories/{id}	Updates a category's details.	Organiser	{ name, distanceKm, maxParticipants, entryFee }	200 OK – updated category · 403 Forbidden · 404 Not Found
DELETE	/api/categories/{id}	Removes a category from an event.	Organiser	None	204 No Content · 403 Forbidden · 404 Not Found · 409 Conflict (existing enrolments)

Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments	Enrols the logged-in participant into a chosen category.	Participant	{ categoryId }	201 Created – enrolment with bib number · 400 Bad Request · 404 Not Found · 409 Conflict (already enrolled / category full)
GET	/api/enrolments/me	Lists all of the logged-in participant's own enrolments, past and upcoming.	Participant	None	200 OK – array of enrolments
DELETE	/api/enrolments/{id}	Cancels the logged-in participant's own enrolment.	Participant	None	204 No Content · 403 Forbidden (not owner) · 404 Not Found

Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/results	Captures a finish time and position for a participant's enrolment.	Organiser	{ enrolmentId, finishTime, position }	201 Created – result object · 400 Bad Request · 403 Forbidden · 404 Not Found · 409 Conflict (result already captured)
PUT	/api/results/{id}	Corrects a previously captured result.	Organiser	{ finishTime, position }	200 OK – updated result · 403 Forbidden · 404 Not Found
GET	/api/results/me	Returns the logged-in participant's personal performance history across all events.	Participant	None	200 OK – array of results
GET	/api/events/{id}/results	Returns the full results list for an event (leaderboard).	Public	None	200 OK – array of results · 404 Not Found

SCREENSHOTS OF EACH TABLE CREATION AND VIEWING

SECTION B

The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the SQL script for creating the 'Users' table. The script includes comments and the following CREATE TABLE statement:

```
CREATE TABLE Users (  
    UserId INT IDENTITY(1,1) PRIMARY KEY,  
    FullName NVARCHAR(100) NOT NULL,  
    Email NVARCHAR(150) NOT NULL UNIQUE,  
    PasswordHash NVARCHAR(255) NOT NULL,  
    Role NVARCHAR(20) NOT NULL,  
    CONSTRAINT CK_Users_Role  
    CHECK (Role IN ('Organiser', 'Participant')),  
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE()  
);
```

The bottom pane shows the 'Results' tab with a table containing 4 rows of data:

	UserId	FullName	Email	PasswordHash	Role
1	1	Sarah Naidoo	sarah.aidoo@raceday.co.za	HASHED_PASSWORD_1	Organise
2	2	Thabo Mokoena	thabo.mokoena@raceday.co.za	HASHED_PASSWORD_2	Organise
3	3	Emma van der Merwe	emma.vdm@example.com	HASHED_PASSWORD_3	Participa
4	4	Sipho Dlamini	sipho.dlamini@example.com	HASHED_PASSWORD_4	Participa

RaceDay.sql ...tudent (52)

```

121  GO
122
123  -- *****
124  -- TABLE: Results
125  -- One result per enrolment (1:1), captured by an Organiser.
126  -- *****
127
128  CREATE TABLE Results (
129      ResultId          INT          IDENTITY(1,1) PRIMARY KEY,
130      EnrolmentId       INT          NOT NULL UNIQUE,
131      FinishTime        TIME         NOT NULL,
132      Position          INT          NULL,
133      CapturedByUserId  INT          NOT NULL,
134      CapturedAt        DATETIME     NOT NULL DEFAULT GETDATE(),
135
136      CONSTRAINT FK_Results_Enrolment
137          FOREIGN KEY (EnrolmentId) REFERENCES EventEnrolments(EnrolmentId),
138
139      CONSTRAINT FK_Results_CapturedBy
140          FOREIGN KEY (CapturedByUserId) REFERENCES Users(UserId)
141  );
142  GO
143
144
145  INSERT INTO Users

```

69 %



1



0



Results



Messages



Client Statistics

	ResultId	EnrolmentId	FinishTime	Position	CapturedByUserId	CapturedAt
1	1	1	00:52:31.0000000	14	1	2026-09-04 21:36:41.407

```

100 -- *****
101 -- TABLE: EventEnrolments
102 -- Links a Participant to a Category they have entered.
103 -- *****
104
105 CREATE TABLE EventEnrolments (
106     EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
107     ParticipantId INT NOT NULL,
108     CategoryId INT NOT NULL,
109     EnrolmentDate DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
110     Status NVARCHAR(20) NOT NULL DEFAULT 'Confirmed',
111     CK_Enrolments_Status
112     CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled')),
113     BibNumber NVARCHAR(10) NULL,
114     CONSTRAINT FK_Enrolments_Participant
115     FOREIGN KEY (ParticipantId) REFERENCES Users(UserId),
116     CONSTRAINT FK_Enrolments_Category
117     FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId),
118     CONSTRAINT UQ_Enrolments_Participant_Category
119     UNIQUE (ParticipantId, CategoryId)
120 );
121 GO
122

```

Saved Change

69 %



1



0



Results



Messages



Client Statistics

	EnrolmentId	ParticipantId	CategoryId	EnrolmentDate	Status	BibNumber
1	1	3	2	2026-09-04	Confirmed	A1023
2	2	4	3	2026-09-04	Confirmed	B2045
3	3	3	5	2026-09-04	Confirmed	C3067

Close (Shift+Esc) : Routes

```

85  -- *****
86  -- route/map information for an event.
87  -- *****
88  CREATE TABLE Routes (
89      RouteId          INT          IDENTITY(1,1) PRIMARY KEY,
90      EventId          INT          NOT NULL,
91      RouteName         NVARCHAR(100) NOT NULL,
92      DistanceKm        DECIMAL(5,2) NOT NULL,
93      ElevationGain     DECIMAL(6,2) NULL,
94      MapUrl            NVARCHAR(255) NULL,
95      CONSTRAINT FK_Routes_Event
96          FOREIGN KEY (EventId) REFERENCES Events(EventId)
97  );
98  GO
99
100  -- *****
101  -- TABLE: EventEnrolments

```

69 %



1



0



Results



Messages



Client Statistics

	RouteId	EventId	RouteName	DistanceKm	ElevationGain	MapUrl
1	1	1	Beachfront Promenade Loop	10.00	25.00	https://raceday.co.za/routes/durban-10k
2	2	2	City Circuit	80.00	410.00	https://raceday.co.za/routes/joburg-cycle
3	3	3	Winelands Trail	5.00	60.00	https://raceday.co.za/routes/stellenbosch-walk

```

65
66 -- ***
67 -- TAE C:\Users\Student\Documents\SQL Server Management Studio 22\RaceD
68 -- Dis (LabVM2049939\Student (52))
69 -- ***
70
71 CREATE TABLE Categories (
72     CategoryId INT IDENTITY(1,1) PRIMARY KEY,
73     EventId INT NOT NULL,
74     Name NVARCHAR(50) NOT NULL,
75     DistanceKm DECIMAL(5,2) NOT NULL,
76     MaxParticipants INT NOT NULL DEFAULT 100,
77     EntryFee DECIMAL(8,2) NOT NULL DEFAULT 0,
78     CONSTRAINT FK_Categories_Event
79         FOREIGN KEY (EventId) REFERENCES Events(EventId)
80 );
81 GO
82
83 -- *****

```

69 %



1



0



Results



Messages



Client Statistics

	CategoryId	EventId	Name	DistanceKm	MaxParticipants	EntryFee
1	1	1	5km	5.00	300	150.00
2	2	1	10km	10.00	500	220.00
3	3	2	40km	40.00	200	350.00
4	4	2	80km	80.00	150	450.00
5	5	3	5km Fun Walk	5.00	400	100.00
6	6	1	5km	5.00	300	150.00
7	7	1	10km	10.00	500	220.00
8	8	2	40km	40.00	200	350.00
9	9	2	80km	80.00	150	450.00
10	10	3	5km Fun Walk	5.00	400	100.00

```

44
45 -- *****
46 -- TABLE: Events
47 -- Each event is created and owned by an Organisier.
48 -- *****
49
50 CREATE TABLE Events (
51     EventId      INT          IDENTITY(1,1) PRIMARY KEY,
52     OrganisierId  INT          NOT NULL,
53     Name         NVARCHAR(150) NOT NULL,
54     Description   NVARCHAR(1000) NULL,
55     EventDate     DATE         NOT NULL,
56     Location      NVARCHAR(150) NOT NULL,
57     EventType     NVARCHAR(20) NOT NULL,
58     CONSTRAINT CK_Events_EventType
59         CHECK (EventType IN ('Running', 'Cycling', 'Walking')),
60     CreatedAt     DATETIME     NOT NULL DEFAULT GETDATE(),
61     CONSTRAINT FK_Events_Organisier
62         FOREIGN KEY (OrganisierId) REFERENCES Users(UserId)
63 );
64 GO
65
66 -- *****
67 -- TABLE: Categories
68 -- Categories are used to group events

```

9 % 1 0 Ln: 260, Ch: 1 (21 chars, 1 lines) SPC

Results Messages Client Statistics

	EventId	OrganisierId	Name	Description	EventDate	Location	EventType	CreatedAt
	1	1	Durban Beachfront 10K	A scenic morning run along the Durban beachfront...	2026-03-15	Durban, KwaZulu-Natal	Running	2026-09-04 21:35:52.590
	2	1	Joburg City Cycle Challenge	A closed-road cycling event through the heart of Jo...	2026-04-12	Johannesburg, Gauteng	Cycling	2026-09-04 21:35:52.590
	3	2	Cape Winelands Fun Walk	A family-friendly walk through the vineyards of Stell...	2026-05-03	Stellenbosch, Western Cape	Walking	2026-09-04 21:35:52.590

FULL SQL CODE

/*

RaceDay Database Script

Part 1 - Section C

*/

-- *****

-- DROP DATABASE IF IT ALREADY EXISTS

-- *****

IF DB_ID('RaceDay') IS NOT NULL

BEGIN

ALTER DATABASE RaceDayDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

DROP DATABASE RaceDayDB;

END

GO

-- *****

-- CREATE DATABASE

-- *****

```
CREATE DATABASE RaceDay;
```

```
GO
```

```
USE RaceDay;
```

```
GO
```

```
-- *****
```

```
-- TABLE: Users
```

```
-- Holds both Organisers and Participants, distinguished by Role.
```

```
-- *****
```

```
CREATE TABLE Users (
```

```
    UserId      INT          IDENTITY(1,1) PRIMARY KEY,
```

```
    FullName    NVARCHAR(100) NOT NULL,
```

```
    Email       NVARCHAR(150) NOT NULL UNIQUE,
```

```
    PasswordHash NVARCHAR(255) NOT NULL,
```

```
    Role        NVARCHAR(20)  NOT NULL,
```

```
    CONSTRAINT CK_Users_Role
```

```
        CHECK (Role IN ('Organiser', 'Participant')),
```

```
    CreatedAt   DATETIME      NOT NULL DEFAULT GETDATE()
```

```
);
```

```
GO
```



```
-- *****
```

```
-- TABLE: Events
```

```
-- Each event is created and owned by an Organiser.
```

```
-- *****
```

```
CREATE TABLE Events (  
    EventId      INT          IDENTITY(1,1) PRIMARY KEY,  
    OrganiserId  INT          NOT NULL,  
    Name         NVARCHAR(150) NOT NULL,  
    Description   NVARCHAR(1000) NULL,  
    EventDate    DATE         NOT NULL,  
    Location     NVARCHAR(150) NOT NULL,  
    EventType    NVARCHAR(20) NOT NULL,  
    CONSTRAINT CK_Events_EventType  
        CHECK (EventType IN ('Running', 'Cycling', 'Walking')),  
    CreatedAt    DATETIME     NOT NULL DEFAULT GETDATE(),  
    CONSTRAINT FK_Events_Organiser  
        FOREIGN KEY (OrganiserId) REFERENCES Users(UserId)  
);  
GO
```

```
-- *****
```

```
-- TABLE: Categories
```

```
-- Distance/entry categories that belong to a single event.
```

```
-- *****
```

```
CREATE TABLE Categories (
```

```
    CategoryId    INT          IDENTITY(1,1) PRIMARY KEY,
```

```
    EventId       INT          NOT NULL,
```

```
    Name          NVARCHAR(50) NOT NULL,
```

```
    DistanceKm    DECIMAL(5,2) NOT NULL,
```

```
    MaxParticipants INT        NOT NULL DEFAULT 100,
```

```
    EntryFee      DECIMAL(8,2) NOT NULL DEFAULT 0,
```

```
    CONSTRAINT FK_Categories_Event
```

```
        FOREIGN KEY (EventId) REFERENCES Events(EventId)
```

```
);
```

```
GO
```

```
-- *****
```

```
-- TABLE: Routes
```

```
-- Route/map information for an event.
```

```
-- *****
```

```

CREATE TABLE Routes (
    RouteId      INT          IDENTITY(1,1) PRIMARY KEY,
    EventId      INT          NOT NULL,
    RouteName    NVARCHAR(100) NOT NULL,
    DistanceKm   DECIMAL(5,2) NOT NULL,
    ElevationGain DECIMAL(6,2) NULL,
    MapUrl       NVARCHAR(255) NULL,
    CONSTRAINT FK_Routes_Event
        FOREIGN KEY (EventId) REFERENCES Events(EventId)
);
GO

```

```

-- *****
-- TABLE: EventEnrolments
-- Links a Participant to a Category they have entered.
-- *****

```

```

CREATE TABLE EventEnrolments (
    EnrolmentId INT          IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT        NOT NULL,
    CategoryId   INT          NOT NULL,
    EnrolmentDate DATE        NOT NULL DEFAULT CAST(GETDATE() AS DATE),

```

```

Status      NVARCHAR(20)  NOT NULL DEFAULT 'Confirmed',
CONSTRAINT CK_Enrolments_Status
    CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled')),
BibNumber    NVARCHAR(10)  NULL,
CONSTRAINT FK_Enrolments_Participant
    FOREIGN KEY (ParticipantId) REFERENCES Users(UserId),
CONSTRAINT FK_Enrolments_Category
    FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId),
CONSTRAINT UQ_Enrolments_Participant_Category
    UNIQUE (ParticipantId, CategoryId)
);
GO

```

```
-- *****
```

```
-- TABLE: Results
```

```
-- One result per enrolment (1:1), captured by an Organiser.
```

```
-- *****
```

```

CREATE TABLE Results (
    ResultId      INT      IDENTITY(1,1) PRIMARY KEY,
    EnrolmentId   INT      NOT NULL UNIQUE,
    FinishTime    TIME      NOT NULL,

```

```

Position      INT      NULL,
CapturedByUserId INT      NOT NULL,
CapturedAt    DATETIME  NOT NULL DEFAULT GETDATE(),

CONSTRAINT FK_Results_Enrolment
    FOREIGN KEY (EnrolmentId) REFERENCES EventEnrolments(EnrolmentId),

CONSTRAINT FK_Results_CapturedBy
    FOREIGN KEY (CapturedByUserId) REFERENCES Users(UserId)
);
GO

INSERT INTO Users
    (FullName, Email, PasswordHash, Role)
VALUES
    ('Sarah Naidoo', 'sarah.naidoo@raceday.co.za',
    'HASHED_PASSWORD_1', 'Organiser'),

    ('Thabo Mokoena', 'thabo.mokoena@raceday.co.za',
    'HASHED_PASSWORD_2', 'Organiser');

-- *****

```

-- Participants

-- *****

INSERT INTO Users

(FullName, Email, PasswordHash, Role)

VALUES

('Emma van der Merwe', 'emma.vdm@example.com',

'HASHED_PASSWORD_3', 'Participant'),

('Sipho Dlamini', 'sipho.dlamini@example.com',

'HASHED_PASSWORD_4', 'Participant');

-- *****

-- Events

-- *****

INSERT INTO Events

(OrganiserId, Name, Description, EventDate, Location, EventType)

VALUES

(1,

'Durban Beachfront 10K',

'A scenic morning run along the Durban beachfront promenade.',

'2026-03-15',

'Durban, KwaZulu-Natal',

'Running'),

(1,

'Joburg City Cycle Challenge',

'A closed-road cycling event through the heart of Johannesburg.',

'2026-04-12',

'Johannesburg, Gauteng',

'Cycling'),

(2,

'Cape Winelands Fun Walk',

'A family-friendly walk through the vineyards of Stellenbosch.',

'2026-05-03',

'Stellenbosch, Western Cape',

'Walking');

-- *****

-- Categories

-- *****

INSERT INTO Categories

(EventId, Name, DistanceKm, MaxParticipants, EntryFee)

VALUES

```
(1, '5km', 5.00, 300, 150.00),  
(1, '10km', 10.00, 500, 220.00),  
(2, '40km', 40.00, 200, 350.00),  
(2, '80km', 80.00, 150, 450.00),  
(3, '5km Fun Walk', 5.00, 400, 100.00);
```

```
-- *****
```

-- Routes

```
-- *****
```

INSERT INTO Routes

```
(EventId, RouteName, DistanceKm, ElevationGain, MapUrl)
```

VALUES

```
(1,  
'Beachfront Promenade Loop',  
10.00,  
25.00,  
'https://raceday.co.za/routes/durban-10k'),
```

```
(2,  
'City Circuit',
```



```
80.00,  
410.00,  
'https://raceday.co.za/routes/joburg-cycle'),
```

```
(3,  
'Winelands Trail',  
5.00,  
60.00,  
'https://raceday.co.za/routes/stellenbosch-walk');
```

```
-- *****
```

```
-- Enrolments
```

```
-- *****
```

```
INSERT INTO EventEnrolments
```

```
(ParticipantId, CategoryId, Status, BibNumber)
```

```
VALUES
```

```
(3, 2, 'Confirmed', 'A1023'), -- Emma enters the Durban 10km
```

```
(4, 3, 'Confirmed', 'B2045'), -- Sipho enters the Joburg 40km
```

```
(3, 5, 'Confirmed', 'C3067'); -- Emma enters the Stellenbosch fun walk
```

```
-- *****
```

```
-- Sample Result
```

```
-- *****
```

```
INSERT INTO Results
```

```
(EnrolmentId, FinishTime, Position, CapturedByUserId)
```

```
VALUES
```

```
(1, '00:52:31', 14, 1);
```

```
GO
```

```
-- *****
```

```
-- DISPLAY ALL TABLES
```

```
-- *****
```

```
USE RaceDay;
```

```
GO
```

```
SELECT * FROM Users;
```

```
SELECT * FROM Events;
```

```
SELECT * FROM Categories;
```

```
SELECT * FROM Routes;
```

```
SELECT * FROM EventEnrolments;
```

```
SELECT * FROM Results;
```

```
GO
```

